

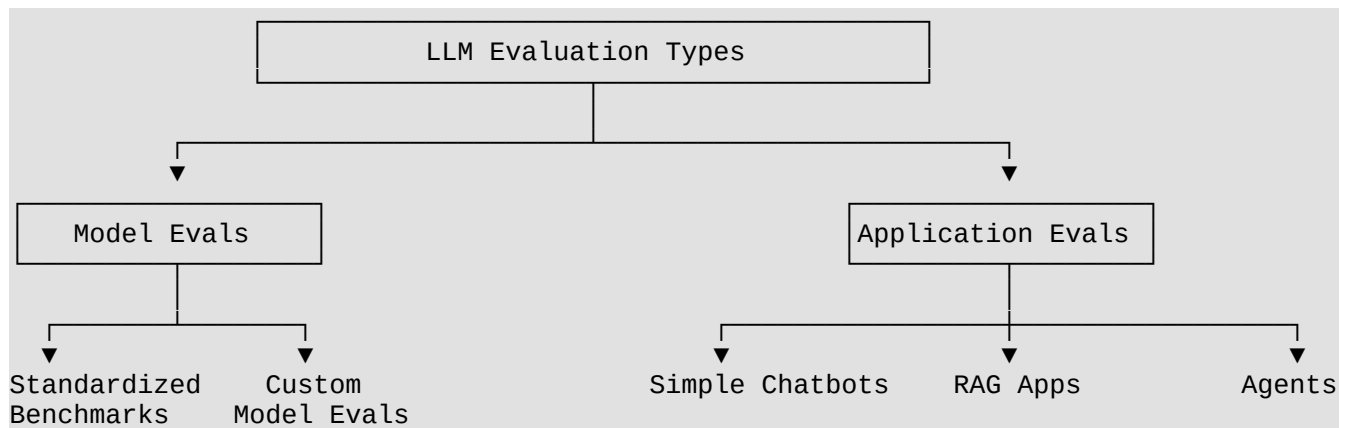
1. Curriculum Recap & Contextual Positioning

Before diving into RAG evaluation, I want to give a quick recap of where we lie in the overall curriculum of this **LLM Evals** course. Up to this point, we have covered two major milestones across our previous lectures:

- **Introduction and Fundamentals:** In our first three to four lectures, we focused on foundational concepts. We established why **LLM Evals** are necessary, defined what they are, and introduced key evaluation dichotomies: **Reference Based Evals** versus **Reference Free Evals**, as well as **Online Evals** versus **Offline Evals**. Crucially, we established the fundamental distinction between two categories of evaluations: **Model Evals** and **Application Evals**.
- **Model Evals:** Although our primary interest lies in Application Evals, we spent the last two to three sessions mastering Model Evals. Within Model Evals, we covered two distinct sub categories:
 - **Standardized Evals (Benchmarks):** Evaluated using standardized benchmarks (such as knowledge capability benchmarks) to test base model capabilities.
 - **Custom Model Evals:** Evaluated by creating your own custom dataset to run evaluations when choosing a specific model tailored to your application's requirements.

Having completed Model Evals, we are now entering the most critical phase of this course:

Application Evals. We will now learn how to evaluate a complete LLM based application using all the foundational knowledge we have gathered so far.



Categorization of LLM Based Applications

There are several distinct architectural paradigms for LLM based applications:

- **Simple Chatbots:** Standard conversational interfaces without external retrieval or tooling.
- **RAG Based Applications:** Applications that combine retrieval systems with generation models.

- **Agents:** Autonomous entities capable of reasoning, planning, and tool execution.
- **Multimodal Applications:** Applications operating across modalities beyond text, such as image generators. These are slightly different in structure and currently less common in production setups.
- **Fixed Schema Output Applications:** Applications where the LLM produces output adhering to a strict, fixed schema. An example is our previously discussed Zomato case study, where an LLM reads support emails and classifies them into structured categories like support, refund or technical emails.

Scope Selection: Why Focus on RAG and Agents?

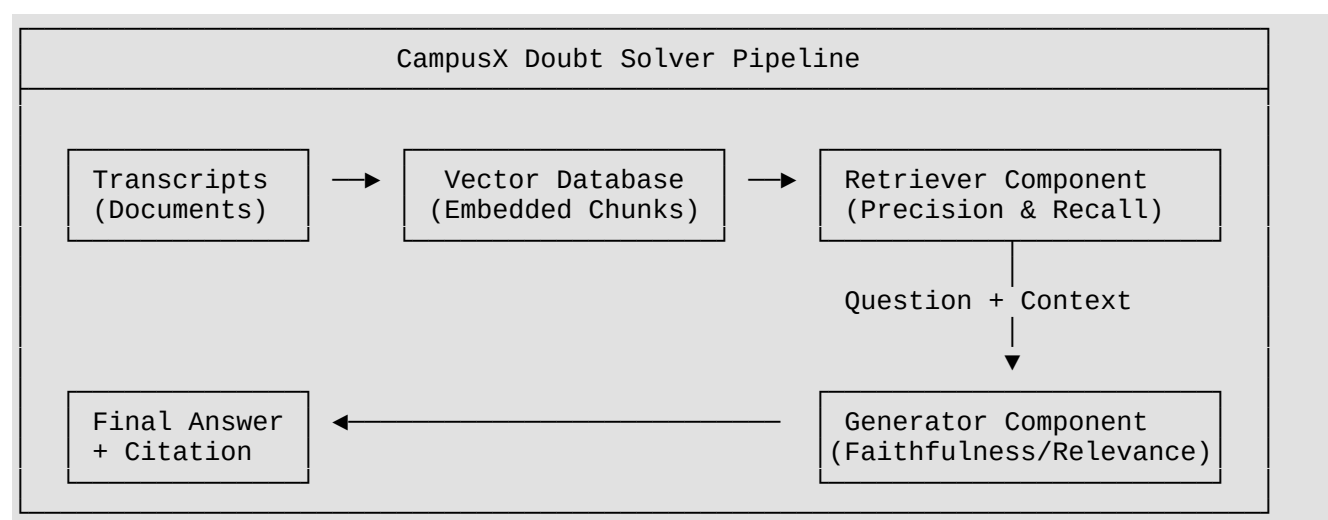
It is impractical to teach specific evaluation suites for every single application variant. Therefore, I have deliberately selected two core application design patterns to cover extensively:

1. **RAG (Retrieval Augmented Generation):** RAG is fundamental because the vast majority of chatbots built in professional industry setups utilize RAG functionality.
2. **Agents:** Agents represent an increasingly vital architectural pattern in modern GenAI development.

If you master evaluations for complex architectures like RAG and Agents, evaluating simpler setups like basic chatbots or fixed schema applications becomes straightforward.

2. Project Case Study Premise: CampusX Doubt Solver

To ground our study of RAG evaluation in practice, we are building and evaluating a real world project: the **CampusX Doubt Solver**.



Problem Statement Definition

- **Core Objective:** Build a specialized RAG chatbot designed to answer student doubts

exclusively for this **LLM Evals** course.

- **Ingested Documents:** In this course, we conduct roughly two lectures per week, providing recordings alongside full **lecture transcripts**. These transcripts serve as our knowledge base documents.
- **Functionality:** The transcripts are processed into vectors and stored in a vector database. Students can query the chatbot regarding any doubt across the playlist, and the system retrieves relevant transcript context to generate answers.
- **Pedagogical Rationale:** Rather than selecting an overly complex or fancy problem statement, I chose a clean, straightforward RAG architecture. This ensures our primary focus remains on mastering **evaluation methodologies** rather than wrestling with application mechanics.

Core Mechanics of the RAG Pipeline

A standard RAG application consists of four primary technical components:

1. **Document Chunks & Vector Database:** Raw text transcripts are split into chunks, passed through an **Embedding Model** and stored as high dimensional vectors inside a **Vector Database**.
2. **Retriever:** When a user submits a query, the Retriever queries the Vector Database to fetch the top (k) most relevant document chunks based on vector similarity.
3. **Generator:** An LLM component that receives the **User Question** combined with the **Retrieved Context** to synthesize a final answer.
4. **Citations:** The generator provides a direct reference link/line to the specific session transcript where the instructor explained the concept.

3. The 3 Level RAG Evaluation Suite Architecture

Evaluation is not a single isolated test performed after an application is completely built. Just as in classical software engineering, where unit and integration tests are executed alongside feature development, building a RAG chatbot and evaluating it must occur concurrently.

We evaluate our RAG application across **three distinct structural levels**:

3 Level Evaluation Architecture		
LEVEL 1: COMPONENT LEVEL EVALUATION		
└ Retriever Evaluation	→	Metrics: Context Precision, Context Recall
└ Generator Evaluation	→	Metrics: Faithfulness, Answer Relevance, Citation Accuracy
LEVEL 2: PIPELINE LEVEL EVALUATION (RAG TRIAD)		
└ Question ↔ Context	→	Metric: Context Relevance
└ Context ↔ Answer	→	Metric: Faithfulness
└ Question ↔ Answer	→	Metric: Answer Relevance

LEVEL 3: APPLICATION / SYSTEM LEVEL EVALUATION

— Quality Metrics	→ Metrics: Correctness, Completeness, Style
— Safety Evals	→ Metrics: Toxicity, PII Leakage, Jailbreak
— Operational Evals	→ Metrics: Latency, Cost per Query, Tokens

Level 1: Component Level Evaluation

In Component Level evaluation, we isolate individual sub systems of the RAG application and verify their performance independently before integrating them.

1. Retriever Component Evaluation (Steps 1 & 2)

- **Development (Step 1):** We write the retriever pipeline code, which handles document loading, text chunking, embedding generation via the embedding model, vector database storage, and query fetching.
- **Isolated Evaluation (Step 2):** We test the retriever in complete isolation from the generator to answer one fundamental question: *Given a user query, does the retriever fetch the correct and relevant document chunks from the vector database?*
- **Retriever Evaluation Metrics:**
 - **Context Recall:** Measures the proportion of all ground truth relevant documents/information that were successfully retrieved from the vector database.
 - **Context Precision:** Measures the proportion of retrieved document chunks that are actually relevant and useful to answering the query.

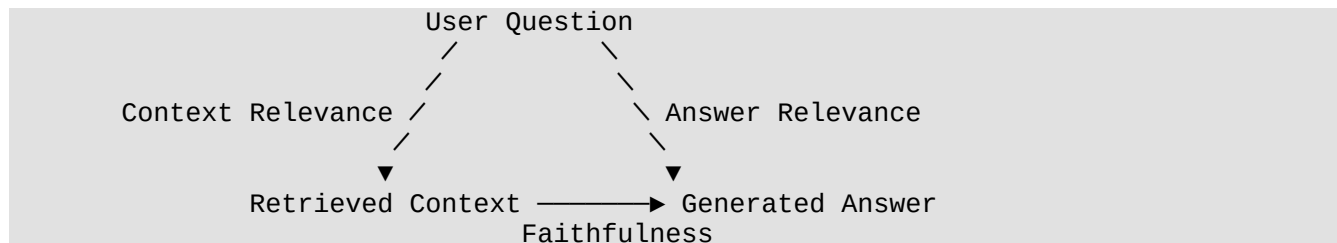
2. Generator Component Evaluation (Steps 3 & 4)

- **Development (Step 3):** We write the generator code, which passes the user query and retrieved context into an LLM to generate an output answer.
- **Isolated Evaluation (Step 4):** We evaluate the generator in isolation without connecting it to the real retriever. We supply manually curated, high quality **Question + Context** pairs directly from a **Golden Dataset**.
- **Generator Evaluation Metrics:**
 - **Faithfulness:** Evaluates whether the generated answer is derived *strictly* and *exclusively* from the provided context, ensuring the model does not hallucinate unmentioned facts.
 - **Answer Relevance:** Evaluates whether the generated response directly addresses the intent of the user query.
 - **Citation Accuracy:** Verifies whether the generator correctly cites the exact session transcript and line where the instructor discussed the concept.

Level 2: Pipeline Level Evaluation & The RAG Triad

Once component evaluations pass, we connect the Retriever and Generator into an integrated **RAG Pipeline** (Step 5) and evaluate their combined performance (Step 6).

Pipeline evaluation centers on the **RAG Triad**, which measures pairwise relationships between the three core textual entities in a RAG run: the **User Question**, the **Retrieved Context**, and the **Generated Answer**.



1. **Context Relevance (Question ↔ Context)**: Evaluates whether the context fetched by the retriever is relevant and tailored to the specific user question.
 2. **Faithfulness (Context ↔ Answer)**: Evaluates whether the final answer generated by the LLM is strictly grounded in the retrieved context without introducing hallucinations.
 3. **Answer Relevance (Question ↔ Answer)**: Evaluates whether the final generated answer directly answers the original question asked by the user.
-

Level 3: Application / System Level Evaluation

After confirming pipeline integrity, we evaluate the system as an end-to-end user facing product across three core dimensions:

1. Quality Related Metrics (Step 7)

- **Correctness**: Evaluates whether the generated answer is factually accurate against ground truth facts.
- **Completeness**: Evaluates whether the response answers *all* parts of a multi part query. If a question contains two sub questions and the model answers only one correctly while omitting the second, the response lacks completeness.
- **Style Alignment**: Evaluates whether the explanation tone and pedagogical style match the specific teaching style of CampusX instructors.

2. Safety Evals (Step 8)

- **Toxicity**: Ensures responses contain no offensive, hostile, or toxic language.
- **PII Leakage**: Ensures no Personally Identifiable Information is inadvertently exposed in outputs.

- **Jailbreak Resistance:** Tests whether the application successfully resists prompt injection or jailbreak attempts designed to bypass system rules.

3. Operational Evals / Ops Evals (Step 9)

- **Latency:** The time taken (in seconds or milliseconds) to generate a complete response per query.
- **Cost:** The monetary cost incurred per query execution based on API token pricing.
- **Token Usage:** The total prompt and completion token counts consumed per transaction.

The combination of all test files across Component, Pipeline, Quality, Safety, and Operational levels constitutes our comprehensive **Eval Suite**.

4. Evaluation Tooling: DeepEval vs. Ragas

Rather than writing low level custom evaluation code for every metric which would balloon the codebase unnecessarily we utilize **DeepEval**, a state of the art open source evaluation framework.

Why DeepEval?

- **Native Metric Support:** DeepEval provides built in implementations for RAG metrics (**Answer Relevancy, Faithfulness, Contextual Precision, Contextual Recall, Contextual Relevancy**) as well as Safety metrics (**Toxicity, PII Leakage**).
- **PyTest Integration:** DeepEval is built directly on top of Python's standard `pytest` testing framework. If you are familiar with software testing in Python, writing DeepEval test cases feels completely natural.

Comparison with Ragas

While **Ragas** is another popular evaluation library, I chose DeepEval for two reasons:

1. **Prior Coverage:** We have already covered Ragas in our Advanced RAG course.
 2. **Broader Functional Scope:** DeepEval supports a wider range of architectures beyond RAG, including LLM Agents, multi turn chatbots, non LLM applications, and image based applications. It is rapidly becoming the industry benchmark standard for LLM evaluation.
-

5. Software Engineering Architecture & Regression Testing

Project Directory Layout

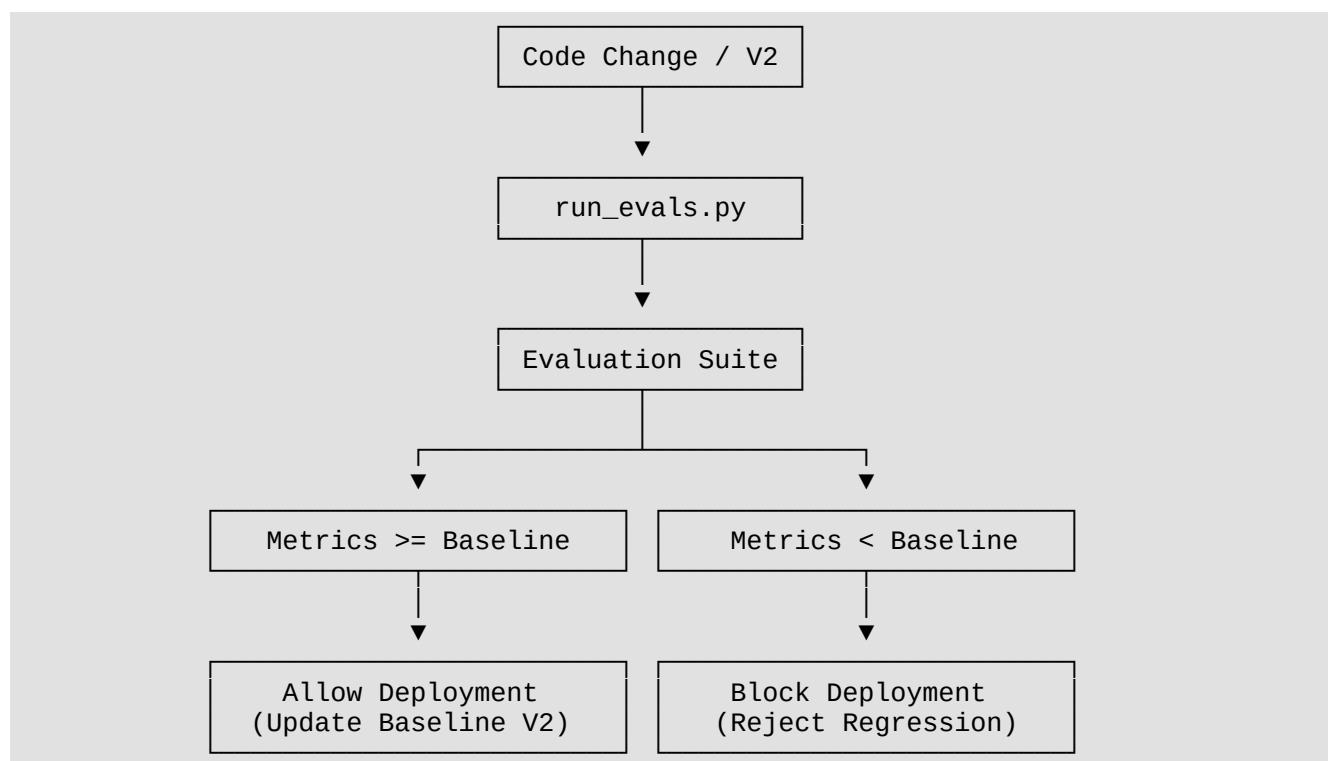
To execute systematic evaluations, we organize our project repository into clear functional modules:

```
campusx_doubt_solver/
```

src/	# Application Source Code
retriever.py	# Document loading, chunking, and retrieval logic
generator.py	# Prompt engineering and LLM generation logic
rag_pipeline.py	# Integrated retriever + generator pipeline
api.py	# FastAPI implementation
app.py	# Streamlit user interface
evals/	# The Complete Eval Suite
eval_retriever.py	# Component tests: Context Precision & Recall
eval_generator.py	# Component tests: Faithfulness & Citation Accuracy
eval_rag_pipeline.py	# Pipeline tests: RAG Triad
eval_application.py	# Application tests: Correctness & Completeness
eval_safety.py	# Safety tests: Toxicity & PII Leakage
eval_ops.py	# Operational tests: Latency & Cost tracking
run_evals.py	# Single root execution runner script

Regression Testing Framework

Regression Testing is the practice of running your entire **Eval Suite** against the application whenever changes are introduced. It ensures that Version 2 (\$V_2\$) of an application is objectively superior to or at least not worse than Version 1 (\$V_1\$) across any metric baseline.



We evaluate regression testing maturity across **three evolutionary levels**:

Level 1: Basic Manual Regression Testing

Executing `run_evals.py` manually triggers all evaluation scripts in `evals/`. The script generates a summary report displaying metric scores (e.g., (Retriever\ Recall = 0.82), (Precision = 0.68)), which developers manually compare against prior run baselines.

Level 2: Automated Experiment Tracking & Dashboards

Integrate experiment tracking tools like **MLflow**, **Confident AI** (by DeepEval), or **Weights & Biases**.

- Hyperparameter configurations (chunk sizes, overlap parameters, LLM temperature settings, embedding models) are logged alongside output metric scores.
- Visual dashboards display historical performance trends across dozens of experiment runs, allowing developers to visually spot performance regressions.

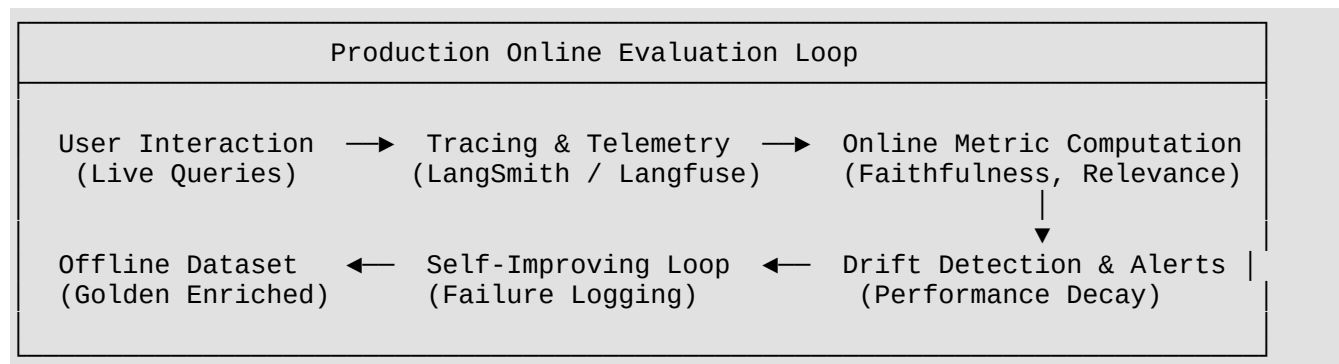
Level 3: CI/CD Automated Gating Mechanisms

Integrate continuous integration tools like **GitHub Actions** directly into the deployment pipeline:

- Every `git push` automatically triggers `run_evals.py` inside a isolated CI workflow.
- **Gating Threshold Rule:** Automated rules compare new metric outputs against current baseline thresholds (e.g., *no metric may degrade by more than 3% relative to the baseline*).
- If metrics fail the threshold, the deployment pipeline automatically halts and blocks the pull request. If metrics pass, the changes are merged, deployed, and the baseline is updated.

6. Post Deployment: Online Evaluations & Observability

Evaluation does not terminate when an application reaches production. Once live, we execute **Online Evals** to continuously monitor system health under real user traffic.



1. Telemetry Capture & Tracing

Using observability platforms like **LangSmith**, **Langfuse**, or **Confident AI**, we embed tracing hooks into production code to log real user interactions:

- Real time Latency (seconds per turn).
- Per query Cost and Token consumption.
- Explicit user feedback signals (e.g., Thumbs Up / Thumbs Down clicks).

2. Computed Production Metrics

We run lightweight online evaluations on production traffic to track metrics like **Faithfulness**, **Answer Relevance**, and **Correctness** in real time.

3. Drift Detection & Alerting

We track metric trends over rolling time windows (e.g., 24-hour moving averages). If a key metric like **Faithfulness** exhibits a sharp downward curve over an 8 hour window, the system flags **performance drift** and triggers alerts for developer intervention.

4. Continuous Self Improving Data Loop

When live production interactions trigger failure modes or poor feedback, those specific interaction traces are captured and ingested back into our offline **Golden Dataset**. This continuously enriches our test dataset, ensuring future offline evaluation suites account for edge cases encountered in production.

7. Practical Implementation Roadmap & Interview Strategy

The 4 Session Practical Execution Roadmap

To put this complete framework into practice, our next four sessions will follow this execution plan:

- **Session 1 (Current Session):** Theoretical architecture, evaluation suite blueprint, regression testing framework, and online evaluation strategy.
 - **Session 2:** Hands on building of the Retriever and Generator components, followed by Component Level evaluation (Context Precision, Recall, Faithfulness, Answer Relevance, and Citations) using DeepEval.
 - **Session 3:** Assembling the full RAG Pipeline and executing Pipeline Level evaluations using the **RAG Triad**.
 - **Session 4:** Implementing Application Level Quality, Safety, and Operational evals, configuring automated **Regression Testing**, and demonstrating **Online Evals** with tracing.
-

How to Master the GenAI Interview Question: "How Do You Evaluate Your RAG App?"

When asked this question in GenAI technical interviews, avoid simply listing three or four metric names. Instead, articulate a structured framework:

1. **3 Level Evaluation Suite:** Explain that you construct a comprehensive **Eval Suite** covering:
 - *Component Level:* Testing the Retriever independently (Context Precision & Recall) and the Generator independently (Faithfulness, Answer Relevance, Citation Accuracy).

- *Pipeline Level*: Evaluating the integrated pipeline using the **RAG Triad** (Context Relevance, Faithfulness, Answer Relevance).
 - *Application Level*: Evaluating overall Quality (Correctness, Completeness, Style Alignment), Safety (Toxicity, PII Leakage, Jailbreak Resistance), and Operations (Latency, Cost, Tokens).
2. **Automated Regression Testing**: Detail how you run this Eval Suite as part of a **Regression Testing** workflow (utilizing tools like MLflow or CI/CD gating via GitHub Actions) to compare metric changes against established baselines before approving deployments.
 3. **Production Online Evals & Feedback Loop**: Describe post deployment monitoring using observability tools (LangSmith/Langfuse) to capture real time telemetry (cost, latency, user feedback), monitor metric drift, and maintain a **Self Improving Data Loop** that routes production failure cases back into the offline golden dataset.